PRACTICAL 11

Aim :

Containerize the React frontend, Express backend, and MongoDB database using Docker and Docker Compose. Configure service networking, correct port mappings, a persistent MongoDB volume, and a single-command full-stack startup.

Code :

backend/Dockerfile :-

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm ci --omit=dev

COPY . .

ENV NODE_ENV=production

EXPOSE 5500

CMD ["node", "server.js"]
```

frontend/Dockerfile :-

```
FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

ARG VITE_API_BASE=http://localhost:5500/api

ENV VITE_API_BASE=$VITE_API_BASE

RUN npm run build

FROM node:20-alpine

WORKDIR /app

COPY --from=build /app ./
```

```
EXPOSE 5173

CMD ["npm", "run", "preview", "--", "--host", "0.0.0.0", "--port", "5173"]
```

Code :

docker-compose.yml :-

```
services:
  mongodb:
    image: mongo:7
    ports:
      - "27017:27017"
    volumes:
      - mongodb_data:/data/db

  backend:
    build: ./backend
    environment:
      PORT: 5500
      MONGO_URI: mongodb://mongodb:27017/awd_task_manager
      JWT_SECRET: awd-practical-11-compose-secret
    ports:
      - "5500:5500"
    depends_on:
      - mongodb

  frontend:
    build:
      context: ./frontend
      args:
        VITE_API_BASE: http://localhost:5500/api
    ports:
      - "5173:5173"
    depends_on:
      - backend

volumes:
  mongodb_data:
```

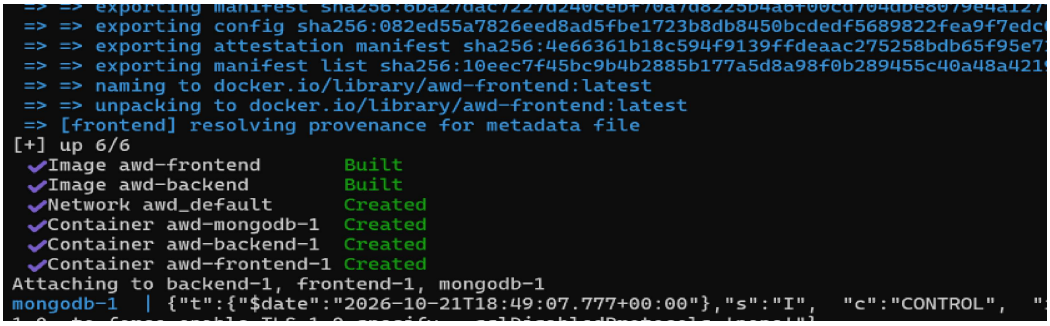
Run command :-

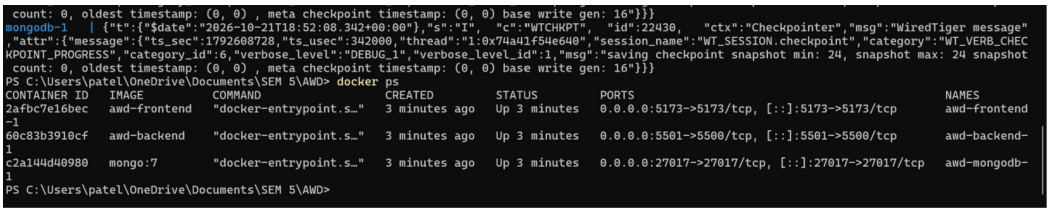
```
docker compose up --build
```

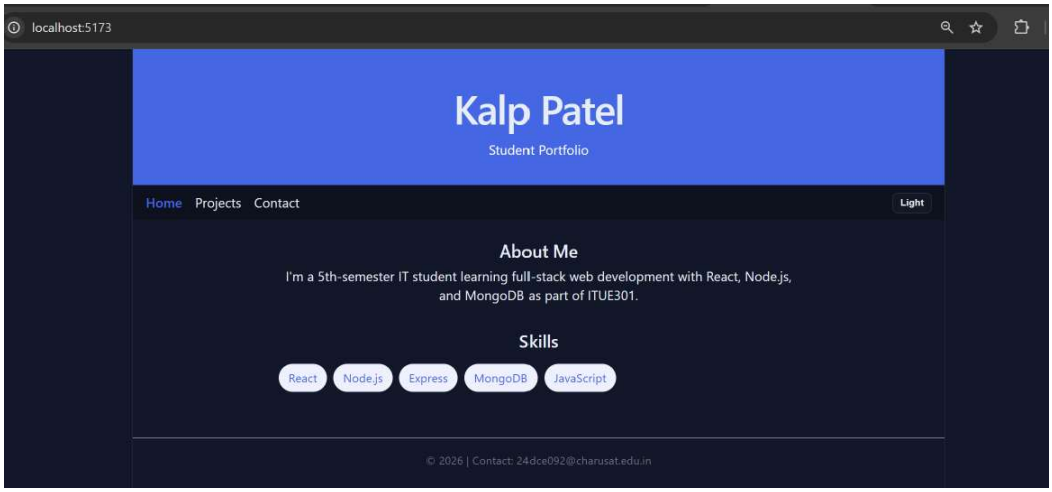
Verify containers :-
docker ps

Output :









Result :

The full-stack application was containerized using Docker and Docker Compose. The frontend, backend, and MongoDB services communicate through the Compose network. MongoDB data is persisted using a named volume, and the complete application can be started with one command.

Important observation :

The backend connects to MongoDB using the Compose service name mongodb, not localhost, because each service runs inside its own container.